Detailed Document on `static` Keyword in Java

The `static` keyword in Java is a modifier that can be applied to variables, methods, blocks, and nested classes. It is used to define members that belong to the class rather than a specific instance of the class. This means the member is shared among all instances of the class and can be accessed without creating an object.

1. `static` Variables

A `static` variable is shared among all instances of a class. It is initialized only once, at the start of the program, and is stored in the class's memory (not instance memory).

Key Points:
- Belongs to the class, not individual objects.
- Commonly used for constants or to maintain a shared state.
- Accessible directly using the class name.

Example:

```
class Counter {
   static int count = 0; // static variable

Counter() {
      count++;
   }

void displayCount() {
      System.out.println("Count: " + count);
   }
}

public class StaticVariableDemo {
   public static void main(String[] args) {
      Counter c1 = new Counter();
      Counter c2 = new Counter();
      Counter c3 = new Counter();

c1.displayCount(); // Output: Count: 3
      c2.displayCount(); // Output: Count: 3
   }
}
```

2. `static` Methods

A `static` method belongs to the class and can be called without creating an instance of the class. Static methods can only access static variables or other static methods directly.

Key Points:
- Cannot use `this` or `super` because they are instance-related.
- Used for utility or helper methods (e.g., `Math` class methods).
- Cannot override but can be hidden in subclasses.

Example:

```
class MathUtil {
   static int square(int x) {
      return x * x;
   }
}

public class StaticMethodDemo {
   public static void main(String[] args) {
      // Calling static method directly using class name
      int result = MathUtil.square(5);
      System.out.println("Square: " + result); // Output: Square: 25
   }
}
```

3. `static` Block

A `static` block is used to initialize static variables or execute code that needs to run only once, before any object is created or static members are accessed.

Key Points:
- Executes when the class is loaded into memory.
- Runs only once during the lifecycle of the application.

Example:

```
class StaticBlockDemo {
   static int num;

static {
      num = 10;
      System.out.println("Static block executed. num initialized to " + num);
   }

public static void main(String[] args) {
      System.out.println("Value of num: " + num);
   }
}
```

Output:


```
Static block executed. num initialized to 10
Value of num: 10
```

4. `static` Inner Classes

An inner class can be made `static`, which means it can be accessed without creating an instance of the enclosing class.

Key Points:
- Does not have access to non-static members of the enclosing class.
- Can access static variables and methods of the outer class.
- Useful for grouping classes logically and when the inner class doesn't need a reference to the outer class.

Example:

```
class Outer {
   static int outerValue = 100;

static class Inner {
      void display() {
         System.out.println("Outer value: " + outerValue);
```

```
    }
  }
}

public class StaticInnerClassDemo {
  public static void main(String[] args) {
    // Creating instance of static inner class
    Outer.Inner inner = new Outer.Inner();
    inner.display(); // Output: Outer value: 100
  }
}
```

Summary of `static` Behaviors

Feature	Behaviors and Restrictions
`static` Variables	Shared among all instances, accessed directly using the class name.
`static` Methods	Can only access static variables and methods directly. Cannot use `this` or `super`.
`static` Block	Executes once when the class is loaded. Used for static initialization.
`static` Inner Classes	Independent of the outer class instance, can access static members of the outer class.

Common Mistakes with `static`

1. **Overusing Static:**
   Avoid using static for everything, as it can lead to tightly coupled code and memory leaks.

2. **Static and Multithreading:**
   Be cautious with static variables in multithreaded environments to avoid race conditions.

3. **Accessing Non-static Members:**
   A static method cannot directly access non-static variables or methods.

Example:

```java
class Example {
  int instanceVar = 10;

static void staticMethod() {
    // Error: Cannot access non-static variable directly
```

```
        // System.out.println(instanceVar);
    }
}
```

Advanced Use Cases

1. **Singleton Pattern:**
   Static is used to create a single instance of a class globally.

2. **Utility Classes:**
   Classes like `Math`, `Collections`, and `Arrays` use static methods to provide global utilities.

3. **Static Import:**
   You can use `static` to import static members of a class to avoid prefixing the class name.
   ```java
   import static java.lang.Math.*;

   public class StaticImportDemo {
       public static void main(String[] args) {
           System.out.println("Square root of 16: " + sqrt(16));
       }
   }
   ```